

Technical Requirements Document

Architecture, stack, security, APIs, algorithms and infrastructure

How Gymholik is built: system architecture, multi-tenancy, security model, backend and mobile design, API conventions and catalogue, key algorithms, deployment on a Fedora server, testing and operations.

Version 1.0 · 8 October 2026

Prepared for: Product owner, Gymholik

Status: Working draft for build and pilot

Confidential. Do not share without permission.

Contents

1. Technical Overview

2. Multi-tenancy Design

3. Security Architecture

4. Backend Design

5. API Catalogue

6. Key Algorithms and Business Logic

7. Mobile Application Design

8. Infrastructure and Deployment

9. Testing Strategy

10. Performance, Scalability and Capacity

11. Architecture Decisions Log

12. Known Technical Risks and Debt

3

4

5

6

8

10

12

13

14

15

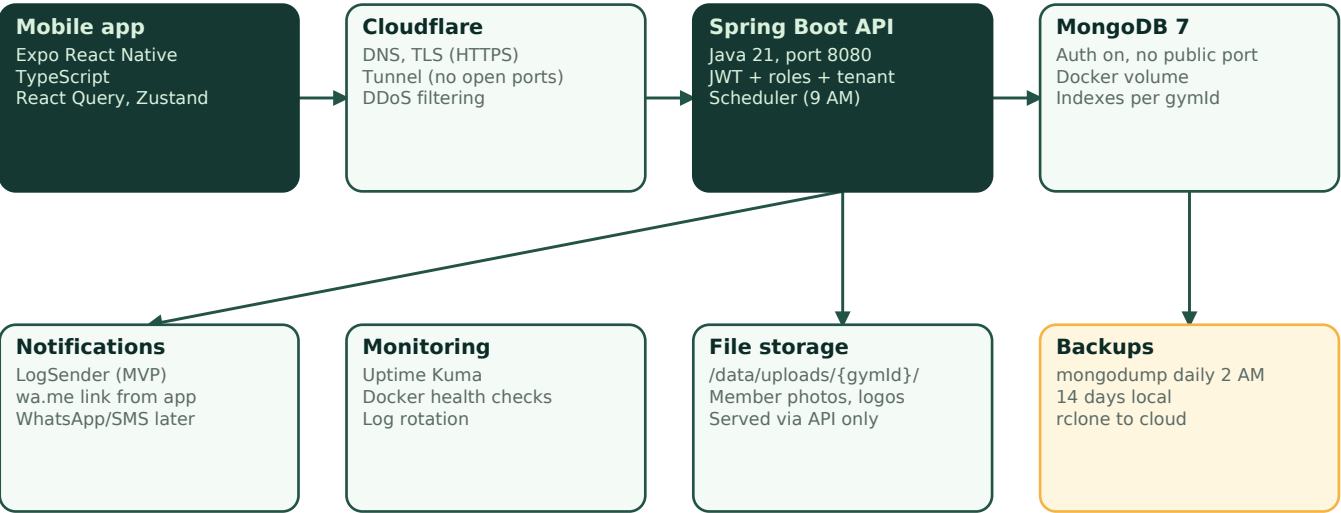
15

16

1. Technical Overview

Gymholik is a client-server system. A React Native (Expo) mobile app talks over HTTPS to a stateless Spring Boot REST API. The API stores data in MongoDB and files on disk. The whole backend runs in Docker Compose on a self-hosted Fedora Linux machine, exposed to the internet through a Cloudflare Tunnel so that no router port needs to be opened.

1.1 System architecture



The server is a home machine (Intel Core i5 8th gen, 16 GB DDR4, Fedora). It is suitable for the pilot and the first 50 to 100 gyms. Because everything runs in Docker, moving to a rented VPS later is a copy of the compose file, the .env file and the latest backup.

1.2 Technology stack

Layer	Technology	Why
Mobile	Expo SDK (current), React Native, TypeScript, expo-router	One codebase for Android and iOS, file-based routing, EAS builds
Mobile state and data	TanStack React Query, Zustand, Axios, React Hook Form, Zod	Caching and pagination, small global store, typed forms with validation
Mobile UX	react-native-reanimated, expo-haptics, expo-blur, expo-image-picker, expo-camera, expo-secure-store, expo-sharing, expo-file-system, react-native-qrcode-svg, react-native-view-shot, a chart library	Animations, camera and QR, secure token storage, sharing receipts and QR cards
Backend	Java 21, Spring Boot 3, Spring Web, Spring Security, Spring Validation, Spring Data MongoDB, Lombok (with SuperBuilder), jjwt, OpenPDF, springdoc-openapi, Bucket4j or similar for rate limiting	Mature stack, strong typing, good security tooling
Database	MongoDB 7 (single node, replica set optional later)	Flexible documents, aggregation pipelines, easy Docker use
Testing	JUnit 5, Mockito, Spring Boot Test, Testcontainers (mongo:7), MockMvc	Real-database integration tests including tenant isolation

Layer	Technology	Why
Infrastructure	Docker, Docker Compose, Fedora Server, Cloudflare Tunnel, Uptime Kuma, rclone, cron	Low cost, easy to move
Dev machines	macOS for development (iOS simulator, Docker Desktop), Fedora for production	Same Docker images in both

Version alignment rule: use the same MongoDB major version everywhere. The integration tests must use mongo:7, the same as docker-compose.yml. A mismatch (for example tests on 6.0 and production on 7) can hide bugs.

2. Multi-tenancy Design

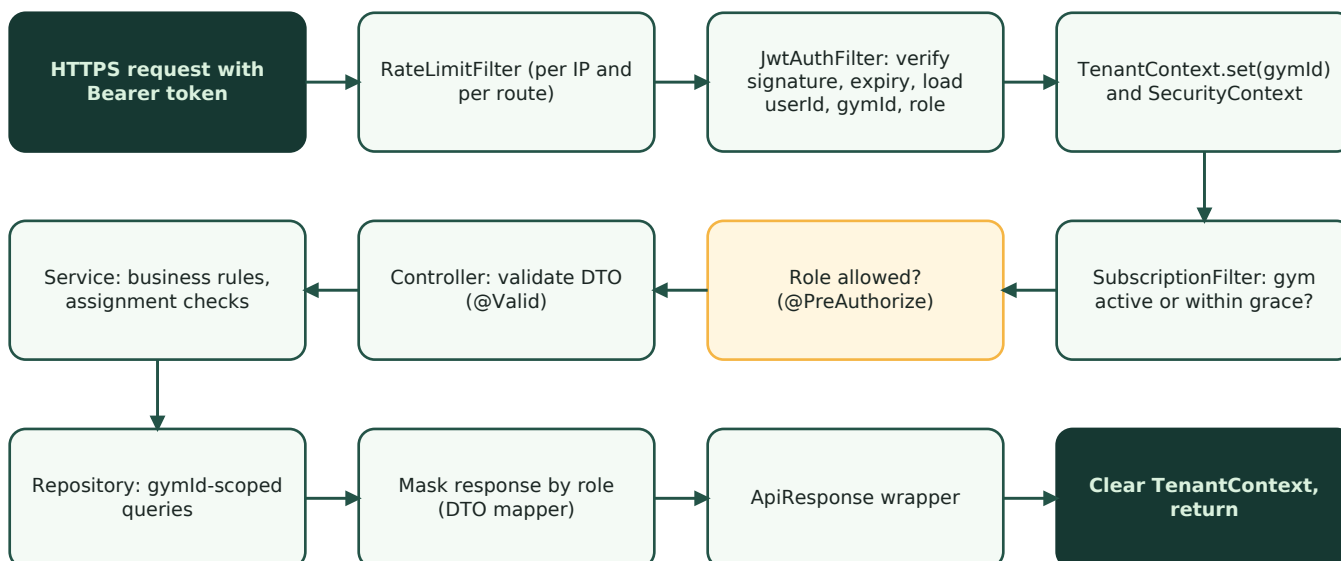
2.1 Model

Shared database, shared collections, tenant discriminator. Every tenant-owned document has a **gymId** field. A single MongoDB database serves all gyms. This keeps operations simple and cost low. The price is that tenant isolation must be enforced consistently in code and verified by tests.

2.2 Rules (non-negotiable)

- gymId is read **only** from the verified JWT and stored in a request-scoped TenantContext. It is never accepted from a request body, path or query string.
- Repositories expose gymId-scoped methods only: findByIdAndGymId, findByGymId and so on. The plain findById is not used on tenant data.
- Every aggregation pipeline starts with a \$match on gymId. Every \$lookup stage must also match on gymId in the joined collection.
- Unique indexes are compound with gymId (for example gymId + phone) except where global uniqueness is required (user login phone, gym code).
- Cross-tenant access returns 404 (not found), not 403, to avoid revealing existence.
- TenantContext is cleared at the end of every request (filter finally block) and in async or scheduled work it is set explicitly per gym.
- A tenant-isolation integration test suite with two gyms must pass before every release.

2.3 Request pipeline



3. Security Architecture

3.1 Authentication

Item	Design
Credentials	Mobile number plus password. Password hashed with BCrypt (cost 10 to 12). Minimum 8 characters with at least one digit.
Access token	JWT, HS256 or RS256, 30 minutes. Claims: sub (userId), gymId, role, tokenVersion. Secret loaded from the JWT_SECRET environment variable, at least 64 random characters.
Refresh token	Opaque random token, 30 days, stored hashed (refreshTokenHash) on the user. Rotated on every refresh. Reuse of an old token invalidates the session.
Lockout	failedAttempts counter. After 5 failures lockedUntil = now + 15 minutes. Counter resets on success.
Rate limits	Login: 10 per minute per IP. Register gym and register trainer: 10 per hour per IP. Gym-code lookup: 10 per hour per IP. Returns HTTP 429.
Client storage	Tokens only in expo-secure-store (Keychain or Keystore). Never AsyncStorage. Logout wipes tokens and the React Query cache.
User states	ACTIVE, PENDING_APPROVAL, REJECTED, INACTIVE. Only ACTIVE users receive tokens.

3.2 Authorisation

- Four roles: SUPER_ADMIN, OWNER, STAFF, TRAINER. Role is a claim in the token and re-checked against the user record when sensitive.
- Method security: @PreAuthorize on every controller method. A test fails the build if any controller method lacks an annotation.
- Service layer repeats ownership checks (for trainers: an ACTIVE assignment must exist). Do not rely on the controller alone.
- SUPER_ADMIN endpoints are under /api/admin and never read gym member data.

3.3 Data masking and least privilege

Field	OWNER	STAFF	TRAINER
name, photo, memberCode	Full	Full	Full (assigned only)
phone	Full	Masked XXXXXX3210 (full if gym.staffCanSeeFullPhone)	Masked always
address, dob	Full	Omitted	Omitted
payments, dues, receipts	Full	Create and limited history	No access (403)
notes	Read all	No access	Assigned only

- Masking is done on the server by role-specific response DTOs (OwnerMemberDto, MaskedMemberDto). The client never receives the full number if it should not see it.
- No tel: or wa.me link is generated in responses for masked roles.
- The same masking applies to QR check-in results, attendance lists, reminder logs, application logs and error messages.
- Logs never contain phone numbers, passwords, tokens or addresses.

3.4 Other protections

Threat	Control
IDOR (guessing IDs)	findByIdAndGymId plus assignment check for trainers, 404 on miss
Mass assignment	Request DTOs have no gymId, role, status, paid amounts or counters

Threat	Control
Injection	Spring Data parameter binding. Never build queries from strings. Escape regex in search or use text/prefix indexes
File upload abuse	Allow jpeg, png, webp only. Max size. Re-encode and compress with ImageIO. Server generates the filename (UUID). Path traversal checks. Served only via an authenticated endpoint that checks gymId
Replay and double submit	Idempotency-Key header on payment creation, stored with a unique index
Gym-code guessing	Rate limit, generic "Invalid gym code" message, owner approval required, owner can regenerate code
Secrets	Only in .env (chmod 600) and environment variables. Not in git. Different secrets for dev and prod
Transport	HTTPS enforced by Cloudflare. API listens on 127.0.0.1 only
CORS	Mobile app does not need CORS. Allow only known web origins if an admin web panel is added
Database exposure	MongoDB has auth on and no published port; only the API container reaches it on the internal Docker network
Dependency risk	Pin versions, run dependency scan before releases

4. Backend Design

4.1 Project structure

```

gym-backend/
  pom.xml Dockerfile docker-compose.yml mongo-init.js CURL_EXAMPLES.md
  src/main/java/com/gymapp/
    GymApplication.java
    config/
      SecurityConfig, MongoConfig (auditing, indexes), OpenApiConfig, WebConfig
    common/
      ApiResponse, ApiError, GlobalExceptionHandler, TenantContext,
      BaseTenantDocument (TenantDocument), Money utils, MaskingUtils, Clock
    auth/
      AuthController, AuthService, JwtService, JwtAuthFilter, RateLimitFilter
    gym/
      Gym, GymRepository, GymService, GymSettingsController, GymCodeGenerator
    user/
      User, UserRepository, StaffController, UserService
    plan/
      Plan, PlanController, PlanService, PlanRepository
    member/
      Member, MemberController, MemberService, MemberRepository, MemberMapper
    membership/
      Membership, MembershipService, MembershipRepository
    payment/
      Payment, PaymentController, PaymentService, ReceiptService (OpenPDF)
    attendance/
      Attendance, AttendanceController, AttendanceService
    trainer/
      TrainerType, TrainerAssignment, MemberNote, TrainerController,
      AssignmentService, NoteService
    dashboard/
      DashboardController, DashboardService (aggregations, 60 s cache)
    reminder/
      ReminderJob (@Scheduled), ReminderLog, NotificationService, LogNotificationSender
    subscription/
      SubscriptionFilter, SubscriptionController, AdminController (SUPER_ADMIN)
    audit/
      AuditLog, AuditService, AuditController
    file/
      FileStorageService, ImageProcessor, FileController
  src/main/resources/application.yml
  src/test/java/... (unit, integration, TenantIsolationIntegrationTest)

```

4.2 Layering and conventions

- Controller (HTTP, validation, role annotation) calls Service (rules, transactions of work) calls Repository (data). DTOs cross the controller boundary; entities do not.
- Lombok: @Getter, @Setter, @SuperBuilder and @NoArgsConstructor on TenantDocument and every entity so builders inherit gymId, and Spring Data can instantiate objects.
- Money: long fields named *Paise. Conversion to rupees happens only in display code.
- Time: LocalDate for business dates (start, expiry, attendance date) interpreted in Asia/Kolkata; Instant for events (paidAt, checkInTime). A Clock bean makes tests deterministic.
- Validation: Jakarta Validation annotations on DTOs; custom validators for Indian mobile numbers (regex `^[6-9]\d{9}$`).

- Exceptions: domain exceptions map to HTTP codes in `GlobalExceptionHandler` (`NotFound` 404, `Conflict` 409, `Validation` 400, `Forbidden` 403, `Unauthorized` 401, `Locked` 423, `TooManyRequests` 429).
- Transactions: MongoDB multi-document transactions need a replica set. In the single-node MVP, operations that must stay consistent (create member + membership + payment) are ordered carefully, use unique indexes and compensation, or MongoDB is started as a single-node replica set so `@Transactional` works. Decision: start Mongo as a single-node replica set (rs0) so transactions are available from day one.

4.3 API conventions

Aspect	Convention
Base path	/api (versioning via /api/v1 is recommended before public release)
Format	JSON, UTF-8. File upload with multipart/form-data. Receipt as application/pdf.
Response wrapper	{ "success": true, "message": "OK", "data": {...}, "error": null }
Error shape	{ "success": false, "message": "...", "data": null, "error": { "code": "SUBSCRIPTION_EXPIRED", "fields": { "phone": "Already registered" } } }
Pagination	Query: page (0-based), size (default 20, max 100). Response: items, page, size, totalItems, totalPages, hasNext.
Sorting and filters	sort=field,direction, status=ACTIVE, from=, to=, search=.
Idempotency	Header Idempotency-Key (UUID) on POST /api/payments.
Dates	ISO 8601: dates as yyyy-MM-dd, instants in UTC with Z.
IDs	MongoDB ObjectId strings.

Standard error codes

Code	HTTP	Meaning
VALIDATION_ERROR	400	Field errors in error.fields
INVALID_CREDENTIALS	401	Wrong mobile number or password
TOKEN_EXPIRED / TOKEN_INVALID	401	Client should refresh or log in again
ACCOUNT_LOCKED	423	Too many failed attempts, retry after 15 minutes
ACCOUNT_PENDING_APPROVAL	403	Trainer waiting for owner approval
ACCOUNT_REJECTED	403	Trainer request declined
SUBSCRIPTION_EXPIRED	403	Gym subscription plus grace has ended
GYM_SUSPENDED	403	Suspended by super admin
FORBIDDEN	403	Role not allowed
NOT_FOUND	404	Missing or belongs to another tenant or not assigned
DUPLICATE_PHONE / DUPLICATE_RESOURCE	409	Unique constraint
ALREADY_CHECKED_IN	409	Attendance already recorded today
OVERPAYMENT	422	Amount exceeds due
MEMBERSHIP_EXPIRED	200 with warning or 403	Depends on gym.allowExpiredCheckin
RATE_LIMITED	429	Too many requests
INTERNAL_ERROR	500	Unexpected. Details only in server logs

5. API Catalogue

Roles: O = OWNER, S = STAFF, T = TRAINER, A = SUPER_ADMIN, P = public. "T*" means trainer limited to assigned members and masked data.

5.1 Auth and account

Method and path	Roles	Purpose
POST /api/auth/register-gym	P	Create gym and owner. Body: gymName, ownerName, phone, password. Starts 14-day trial. Returns tokens.
POST /api/auth/register-trainer	P	Body: gymCode, name, phone, password. Creates TRAINER with PENDING_APPROVAL.
GET /api/auth/gym-code/{code}	P	Returns only gym name for the "Joining: ..." hint. Rate limited.
POST /api/auth/login	P	Body: phone, password. Returns accessToken, refreshToken, user, gym summary.
POST /api/auth/refresh	P	Rotate refresh token.
POST /api/auth/logout	All	Invalidate refresh token.
GET /api/me	All	Current user, role, gym name, trainer types, subscription banner data.
POST /api/me/change-password	All	Old and new password.

5.2 Gym, staff, trainers, subscription

Method and path	Roles	Purpose
GET, PUT /api/gym/settings	O	Name, address, logo, allowExpiredCheckin, reminderDaysBefore, staffCanSeeFullPhone.
GET /api/gym/code	O	Current gym code.
POST /api/gym/code/regenerate	O	New code, old one invalid.
GET, POST /api/staff	O	List, create front-desk staff.
PATCH /api/staff/{id}	O	Activate, deactivate, reset password.
GET /api/trainers	O	List trainers with types, status, assigned count.
GET /api/trainers/pending	O	Pending requests.
POST /api/trainers/{id}/approve	O	Body: trainerTypeIds (required, at least one).
POST /api/trainers/{id}/reject	O	Reject request.
PATCH /api/trainers/{id}	O	Change types, deactivate (ends assignments).
GET, POST, PUT, DELETE /api/trainer-types	O	Manage types (DELETE = deactivate).
GET /api/subscription	O	Plan, validTill, daysLeft, status.

5.3 Plans and members

Method and path	Roles	Purpose
GET /api/plans	O, S	Active plans (used by pickers).
POST, PUT /api/plans, PATCH /api/plans/{id}	O	Create, edit, deactivate.
GET /api/members?search=&status=&page=&size=	O, S, T*	Paginated list. Trainer sees assigned only, masked.
POST /api/members	O, S	Create member with first membership and optional first payment. Multipart or JSON plus photo endpoint.
GET /api/members/{id}	O, S, T*	Detail. Response DTO depends on role.

Method and path	Roles	Purpose
PUT /api/members/{id}	O, S	Edit.
DELETE /api/members/{id}	O	Soft delete.
POST /api/members/{id}/photo	O, S	Upload and compress photo.
GET /api/members/{id}/photo	O, S, T*	Authenticated photo stream.
POST /api/members/{id}/renew	O, S	Body: planId, startDate (optional), optional payment.
GET /api/members/{id}/memberships	O, S	Membership history.
GET /api/members/{id}/qr	O, S	Member code payload for QR card.

5.4 Payments and dues

Method and path	Roles	Purpose
POST /api/payments	O, S	Body: memberId, membershipId, amountPaise, mode, note. Header Idempotency-Key.
GET /api/payments?from=&to=&mode=&page=	O, S (limited)	History plus summary by mode.
GET /api/members/{id}/payments	O, S	Member payment history.
GET /api/payments/{id}/receipt	O, S	PDF receipt.
POST /api/payments/{id}/reverse	O	Body: reason (required). Creates a negative REVERSAL entry.
GET /api/dues	O, S	Members with pending dues, sorted by amount desc.

5.5 Attendance

Method and path	Roles	Purpose
POST /api/attendance/checkin	O, S, T*	Body: memberId or memberCode. Returns result: CHECKED_IN, ALREADY_CHECKED_IN, EXPIRED_WARNING or EXPIRED_BLOCKED.
GET /api/attendance?date=	O, S, T*	Day list with count.
GET /api/members/{id}/attendance?days=30	O, S, T*	History.
GET /api/attendance/absent?days=7	O, S, T*	Members not seen for N days, including never visited.

5.6 Trainer assignments and notes

Method and path	Roles	Purpose
POST /api/assignments	O	Body: trainerId, memberIds[], ptFeePaise, sessionsTotal. Bulk assign.
GET /api/assignments?trainerId=&memberId=&status=	O	List.
POST /api/assignments/{id}/end	O	End assignment.
POST /api/assignments/reassign	O	Body: fromTrainerId, toTrainerId (all active members).
GET /api/my/members	T	Assigned members, masked.
GET, POST /api/members/{id}/notes	T* (read and write), O (read)	Workout, diet, progress, general notes.
PUT /api/notes/{id}	T (own notes)	Edit note.
GET /api/members/{id}/progress	T*, O	Weight and measurement series for charts.

5.7 Dashboard, audit and admin

Method and path	Roles	Purpose
GET /api/dashboard	O (full), S (no money), T (none)	Cards, expiring soon, 7-day series, subscription days left.
GET /api/audit-logs?page=	O	Paginated audit entries.
GET /api/admin/gyms?status=&page=	A	All gyms with status and validity.
POST /api/admin/gyms/{id}/extend	A	Body: plan MONTHLY or YEARLY (or days).
POST /api/admin/gyms/{id}/suspend and /reactivate	A	Control access.

6. Key Algorithms and Business Logic

6.1 Create member with first membership and payment

1. Validate DTO. Check phone is unique within gymId (unique index is the final guard).
2. Load plan with findByIdAndGymId and verify it is active.
3. Generate memberCode using the atomic counter (key member:{gymId}), for example GYM-0045.
4. Create Member. Create Membership: startDate, expiryDate = startDate + plan.durationDays, totalPaise = plan.pricePaise, paidPaise = 0, duePaise = totalPaise.
5. If a first payment is provided: validate paidNow <= total, generate receipt number, create Payment, update membership paid and due.
6. Update denormalised fields on Member: currentExpiry, currentDuePaise.
7. Write audit log. Return the masked or full DTO according to caller role.

Steps 3 to 6 run inside one MongoDB transaction (single-node replica set). If anything fails, nothing is saved.

6.2 Renewal

```
prev = latest membership of member (by startDate desc)
start = (prev.expiryDate < today) ? today : prev.expiryDate // explicit startDate allowed within limits
expiry = start + plan.durationDays
prev.status = EXPIRED when prev.expiryDate < today
new Membership(planId, planName, start, expiry, total = plan.pricePaise, paid = 0, due = total)
member.currentExpiry = expiry; member.currentDuePaise += total
```

6.3 Payment

```
POST /api/payments (Idempotency-Key: K)
if payments.exists(gymId, idempotencyKey = K): return the stored result
membership = findByIdAndGymId(membershipId) // 404 if other tenant
if amountPaise <= 0 or amountPaise > membership.duePaise: reject OVERPAYMENT
receiptNo = counter.next("receipt:" + gymId + ":" + year) // atomic findAndModify $inc
insert Payment(type = PAYMENT, receiptNo, paidAt = now, receivedBy = userId)
membership.paidPaise += amount; membership.duePaise -= amount
member.currentDuePaise = sum of due across active memberships
audit(PAYMENT_CREATED)
```

Reversal creates a second Payment with type REVERSAL, amountPaise negative, reversalOfId set and a mandatory reason. The membership due is increased again. The original payment is never edited or deleted.

6.4 Attendance check-in

1. Resolve member by id or memberCode within gymId. If caller is TRAINER, require an ACTIVE assignment, else 404.
2. If member.status is INACTIVE: reject.
3. Compare member.currentExpiry with today. If expired: if gym.allowExpiredCheckin is false return EXPIRED_BLOCKED, else continue with expiredWarning = true.
4. Insert Attendance(gymId, memberId, date = today). The unique index (gymId, memberId, date) rejects duplicates and the service maps that to ALREADY_CHECKED_IN.
5. Set member.lastAttendanceDate = today. Return result with the masked member summary.

6.5 Absent members

Query members where gymId matches, status is ACTIVE, and lastAttendanceDate is null or older than N days. Because lastAttendanceDate is denormalised on Member, this is a single indexed query instead of an aggregation over all attendance. For trainers add the condition memberId in (assigned member ids).

6.6 Dashboard aggregations

Metric	Computation
activeMembers	count members where currentExpiry >= today and status ACTIVE
todayCollection	sum amountPaise of payments between start and end of today (Asia/Kolkata), including reversals as negatives
monthCollection	same for the calendar month
expiringSoon	members where currentExpiry between today and today + 5, sorted ascending, with days left
newJoinings	count members with joiningDate in this month
todayAttendance	count attendance where date = today
totalDue	sum of currentDuePaise for active members
last7Days	payments grouped by date over 7 days, zero-filled
subscription	validTill, daysLeft, banner flag when 7 days or less

The result is cached per gym for 60 seconds and evicted when a payment, member or attendance is written.

6.7 Daily reminder job

```
@Scheduled(cron = "0 0 9 * * **", zone = "Asia/Kolkata")
for each gym with status in (TRIAL, ACTIVE) and not suspended:
  TenantContext.set(gym.id)
  for type, offset in [("D3", 3), ("D1", 1), ("D0", 0)]:
    memberships = find ACTIVE where expiryDate == today + offset
    for m in memberships:
      try insert ReminderLog(gymId, membershipId, type)    // unique index
      catch duplicate: continue
      notificationService.send(memberSummary, template(type))
  TenantContext.clear()
```

6.8 Trainer visibility

```
visibleMemberIds(trainerId) = distinct memberId from trainer_assignments
                             where gymId = ? and trainerId = ? and status = ACTIVE
list members    : members where gymId = ? and _id in visibleMemberIds
member by id    : if id not in visibleMemberIds -> NotFound (404)
notes, attendance, progress: same check first
```

6.9 Receipt and member code generation

Counters live in the counters collection. A call such as findAndModify with \$inc and upsert returns the new value atomically, so concurrent requests never get the same number. Keys: member:{gymId} and receipt:{gymId}:{year}.

6.10 Gym code generation

Random 6 to 8 characters from an alphabet without look-alike characters (no 0, O, 1, l, L). Retry on collision (unique index). Owner can regenerate; the previous code stops working immediately.

7. Mobile Application Design

7.1 Project structure

```

gym-app/
  app/
    expo-router screens
    _layout.tsx      providers, fonts, auth gate, error boundary
    (auth)/ login, register, subscription-expired
    (tabs)/ _layout.tsx (role tabs), index (Home), members/, checkin, payments/,
              notes, profile, more/
    members/[id].tsx, members/add.tsx, members/[id]/renew.tsx
    trainers/ index, [id], types, assign
    settings/ gym, staff, plans, audit, subscription, change-password
  src/
    api/      axios instance, interceptors, endpoints per module, generated types
    components/ Screen, GlassCard, Button, Input, Badge, Chip, Avatar, Skeleton,
                  EmptyState, ErrorState, Toast, FloatingTabBar, PlanPicker, PhotoPicker
    navigation/ tabsByRole.ts (single source of truth), FloatingTabBar.tsx
    store/      auth store (user, role, tokens state), ui store
    hooks/      useDebounce, useRole, useRefreshOnFocus
    theme/      tokens (colors, spacing, radius, type), fonts
    strings/    en.ts (all user-facing text)
    utils/      money (paise to rupees en-IN), dates, masking helpers, validators

```

7.2 Key mobile decisions

Topic	Decision
Navigation	expo-router. A single tabsByRole config decides which tabs exist. Hidden routes use href: null and a guard redirects deep links.
Role source	Role comes only from the authenticated user object returned by login or /me. The tab layout waits for it before rendering (no flash of wrong tabs).
Tabs per role	OWNER and STAFF: Home, Members, Check-in (raised centre), Payments, More. TRAINER: My Members, Check-in (raised centre), Notes, Profile.
Networking	One axios instance. Request interceptor adds the Bearer token. Response interceptor refreshes once on 401 then retries; on failure logs out. Errors are classified (network, validation, auth, locked, subscription, server) and mapped to user-friendly strings.
Server state	React Query with staleTime 60 s for dashboard and lists; invalidate related keys after writes (members, dashboard, payments).
Forms	React Hook Form with Zod schemas; server field errors mapped to fields; submit disabled while loading to prevent double submits.
Money	API sends paise. Display with Intl en-IN and the ₹ sign only in the UI layer.
Security	expo-secure-store for tokens; clear query cache on logout; no member data written to disk; screens for forbidden roles are not mounted.
Environment	EXPO_PUBLIC_API_URL for the base URL. Simulator: http://localhost:8080. Real phone on Wi-Fi: Mac LAN IP. Production: https://api.yourdomain.com. Restart Expo with -c after changing env.
Types	Generate TypeScript types from the backend OpenAPI JSON (openapi-typescript) to prevent field name drift.

Topic	Decision
Performance	FlatList with keyExtractor, memoised rows, windowing. Animations through reanimated worklets. Blur used sparingly; fallback to solid translucent cards on low-end Android. Respect reduce-motion.
Distribution	EAS Build for Android APK (preview) and AAB (production). iOS later.

7.3 Screen and data map

Screen	Main API calls	Query keys invalidated on write
Login / Register	POST /auth/login, /auth/register-gym, /auth/register-trainer, GET /auth/gym-code/{code}	-
Home	GET /dashboard	dashboard
Members list	GET /members	members
Add member	GET /plans, POST /members, POST /members/{id}/photo	members, dashboard, payments
Member detail	GET /members/{id}, /attendance, /payments, /notes	member:{id}
Collect payment	POST /payments (idempotent), GET receipt	member:{id}, payments, dues, dashboard
Check-in and scan	POST /attendance/checkin, GET /attendance	attendance, dashboard
Trainers (owner)	GET /trainers, approve, assign	trainers, assignments
Notes (trainer)	GET and POST /members/{id}/notes	notes:{memberId}

8. Infrastructure and Deployment

8.1 Server

Item	Value
Hardware	Intel Core i5 (8th gen), 16 GB DDR4 RAM, SSD recommended
OS	Fedora Server (latest), SELinux enforcing, firewalld on
Runtime	Docker Engine and Compose plugin
Network	Cloudflare Tunnel (cloudflared as a systemd service). No inbound ports. SSH only on the local network or key-based.
Power	UPS strongly recommended. A power cut means every gym is offline.
Domain	api.yourdomain.com routed through the tunnel to http://localhost:8080

8.2 Docker Compose services

Service	Image	Notes
mongo	mongo:7	Root user from .env. Volume ./data/mongo:/data/db:Z. No ports published. Started as replica set rs0 (single node). Healthcheck.
api	Built from backend/Dockerfile (Temurin 21 JRE, non-root user)	Port 127.0.0.1:8080:8080. Volume ./data/uploads:/data/uploads:Z. -Xmx768m. Reads env vars. depends_on mongo healthy.
kuma (optional)	louislam/uptime-kuma:1	Port 127.0.0.1:3001 only. Alerts when API health fails.

8.3 Environment variables

Variable	Purpose
MONGO_ROOT_USER, MONGO_ROOT_PASSWORD	MongoDB root credentials
SPRING_DATA_MONGODB_URI	mongodb://user:pass@mongo:27017/gymdb?authSource=admin&replicaSet=rs0
JWT_SECRET	64 or more random characters (openssl rand -base64 48). Application must refuse to start if missing
UPLOAD_DIR	/data/uploads
SPRING_PROFILES_ACTIVE	prod or dev
APP_TIMEZONE	Asia/Kolkata
RATE_LIMIT settings	Per-route limits

A missing JWT_SECRET prevents the Spring context from loading. This is the most likely cause of "Failed to load ApplicationContext" in tests and on first start. Provide a test secret in src/test/resources/application-test.yml.

8.4 Backup and restore

- Nightly 2 AM cron: mongodump --archive --gzip of gymdb, file named gymdb-YYYY-MM-DD.gz.
- Keep 14 days locally in data/backups. Copy off-site with rclone (Google Drive, Backblaze B2 or similar). Also back up data/uploads (photos).
- Restore script with mongorestore --archive --gzip --drop into a scratch database first. Test a restore monthly and record the result.
- Never keep the only copy on the same machine.

8.5 Monitoring and logging

- Spring Actuator health endpoint (internal) used by Docker healthcheck and Uptime Kuma.
- Structured logs without personal data. Docker log rotation (max-size 10m, max-file 5).
- Disk space alert at 80 percent. Weekly check of backup logs.
- Optional: Sentry (free tier) for mobile crash and API errors.

8.6 Moving to a VPS

Trigger: 10 or more paying gyms, or repeated outages. Steps: provision a VPS (about ₹500 to 800 per month), install Docker, copy compose files and .env, restore the latest backup, point the Cloudflare Tunnel or DNS to the new host, verify with the smoke test, then retire the home server. Expect about one hour of work and a short maintenance window.

9. Testing Strategy

Level	Tools	What is covered
Unit	JUnit 5, Mockito	Services: expiry maths, renewal start, due calculation, overpayment, masking utility, gym code generator
Integration	SpringBootTest, MockMvc, Testcontainers mongo:7 (singleton container)	Controllers with real database, security rules, unique indexes, idempotency, counters under concurrency
Tenant isolation	Two gyms (A and B), each with owner, staff, trainer, members, payments, attendance	A cannot read, update, delete or list any B resource by id, search, dashboard, dues, absent or receipt (expect 404). Staff and trainer get 403 on owner endpoints

Level	Tools	What is covered
Trainer access	Same harness	Unassigned member 404, assigned member visible, full phone never in the JSON, no payment endpoints, ending assignment removes access, deactivation ends assignments
Subscription	Time-controlled Clock	Trial, valid, within grace, expired, suspended behaviours
Security	Review prompts and checklists	IDOR, mass assignment, upload safety, log PII, JWT handling
Mobile	Manual on real devices, later Jest and Detox or Maestro	Role tabs for three accounts, forms, offline banner, low-end Android performance
Load (smoke)	k6 or Gatling, small	50 concurrent users on list and check-in; verify response times and no errors

Release gate: mvn clean test must be fully green, including TenantIsolationIntegrationTest, before any deployment. Never disable or weaken these tests to get a green build. The tests require Docker running (Docker Desktop on the Mac).

10. Performance, Scalability and Capacity

- Expected load per gym: a few hundred members, tens of check-ins and payments per day. Even 100 gyms is small for this hardware.
- Indexes support every query path (see Backend Schema document). Dashboard aggregation is cached 60 seconds.
- Lists are paginated; search uses prefix or text indexes and escapes input.
- Denormalised fields (currentExpiry, currentDuePaise, lastAttendanceDate) avoid joins on hot screens. They are updated in the same transaction as the source change.
- JVM heap capped at 768 MB per container. Mongo WiredTiger cache limited to about 2 GB.
- Photos are compressed to under about 500 KB and max 800 px wide. Serve with cache headers.
- Scaling path: vertical (VPS) first, then MongoDB replica set, then horizontal API instances behind a load balancer (the API is stateless).

11. Architecture Decisions Log

ID	Decision	Reason	Trade-off
ADR-01	Shared collections with gymId	Simple, cheap, easy backups	Needs strict discipline and isolation tests
ADR-02	Login with phone only, gym resolved from user	Better UX than asking a gym code at login	Phone must be globally unique
ADR-03	Separate TRAINER role (not STAFF)	Different permissions and data visibility	One more role to maintain
ADR-04	Server-side masking with role DTOs	Real privacy, not cosmetic	More DTO classes
ADR-05	Money as long paise	No rounding errors	Conversion in UI
ADR-06	No hard delete of members and payments	Audit and money history	Storage grows slowly
ADR-07	404 for unassigned or cross-tenant ids	Do not reveal existence	Slightly harder debugging; use server logs
ADR-08	Mongo single-node replica set	Enables transactions	Slightly more setup
ADR-09	wa.me links in MVP, API automation later	No DLT or template approvals needed to launch	Owner taps to send
ADR-10	Cloudflare Tunnel for access	Works behind CGNAT, free HTTPS, hides IP	Dependency on Cloudflare

ID	Decision	Reason	Trade-off
ADR-11	Trainer sees only owner-assigned members	Privacy and owner control	Owner must assign manually
ADR-12	Single source of truth files (tabsByRole, theme, strings)	Prevents drift and role bugs	Slightly more upfront structure

12. Known Technical Risks and Debt

Item	Detail	Action
TenantIsolationIntegrationTest failing to load context	Likely missing JWT_SECRET in test config, Mongo URI not injected from Testcontainers, or index conflict after entity changes	Read the last "Caused by" in target/surefire-reports, fix, keep assertions intact
Mongo version mismatch	Tests used 6.0, production is 7	Use mongo:7 constant shared by all tests
Role tabs bug in mobile	Owner sees trainer tabs; seven tabs shown	Implement tabsByRole and the custom tab bar
Logo with fake transparent background	Checkerboard visible	Replace with real transparent PNG or SVG
Safe area and bottom padding	Content hidden behind tab bar	Shared Screen component with insets and tab bar height
Transactions on standalone Mongo	Not available without a replica set	Run single-node replica set
Third-party images	Brand logos visible in the background photo	Heavy overlay or replace before Play Store release